

Lab 2

Batch and Near Real-Time Data Pipelines with Node-RED and Grafana

Course Number and Name: SEP 6DA3: Data Analytics and Big Data	
Semester, Year, and Group Number: 2025 Fall, Group 6	
Name of Students: Andi Dong Zhiyu Hu Foram Brahmbhatt Jiarui Yang Linghe Shen Shannon Chen	Name of Instructor and TA: Pedro Tondo Muskan Sidana

1. Objective	3
2. Node-RED Flows	3
2.1 Batch Pipeline	3
2.2 Streaming Pipeline	4
3. Grafana Dashboards	4
3.1 Batch Pipeline	4
3.2 Streaming Pipeline	7
3.3 Results Overview.....	9
4. Bonus.....	10
4.1 Prediction Stage	10
4.2 Aggregation by Season and Weather	11
5. Write-up (Main Analysis)	12
5.1 What are the differences between batch vs streaming pipelines?	12
5.2 What issues came up?	13
5.3 Two suggestions for improvement	13
6. Technical Issues & Solutions	14
7. Reflection	17

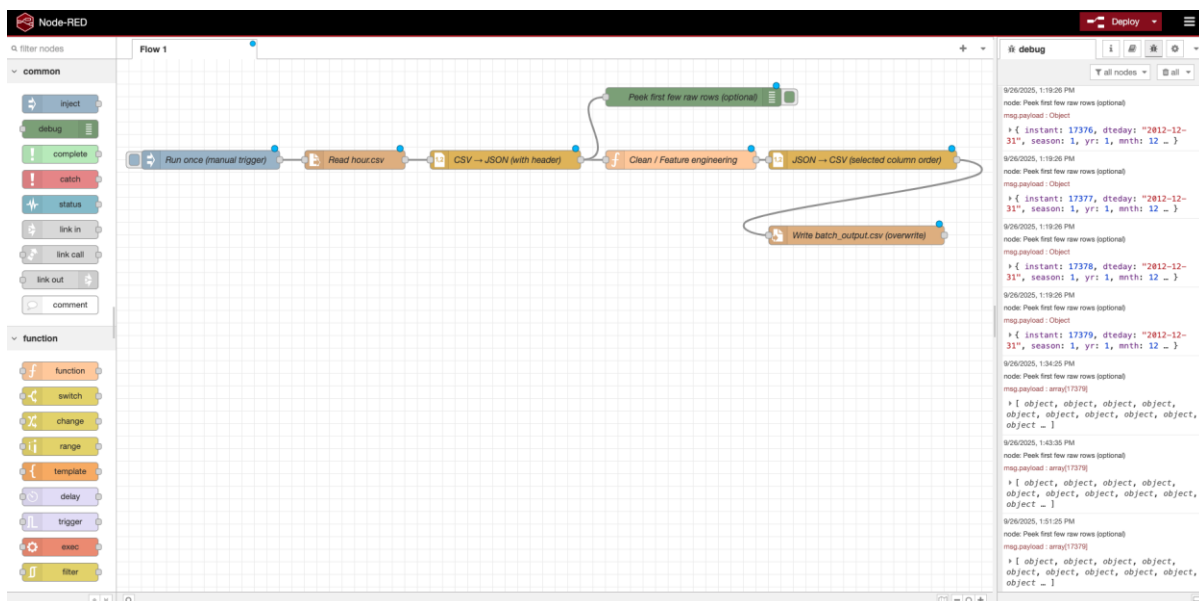
1. Objective

The objective of this assignment is to design and implement two types of data pipelines using Node-RED and visualize the results in Grafana. The first pipeline is a **batch pipeline**, which processes the entire dataset (hour.csv) at once, applies cleaning and transformations, and outputs a structured file for historical analysis. The second pipeline is a **streaming pipeline**, which simulates near real-time data flow by appending new records every few seconds, enabling continuous visualization of live data. Through this work, the goal is to gain hands-on experience with both batch and streaming data processing paradigms, and to understand their roles in data engineering and analytics.

2. Node-RED Flows

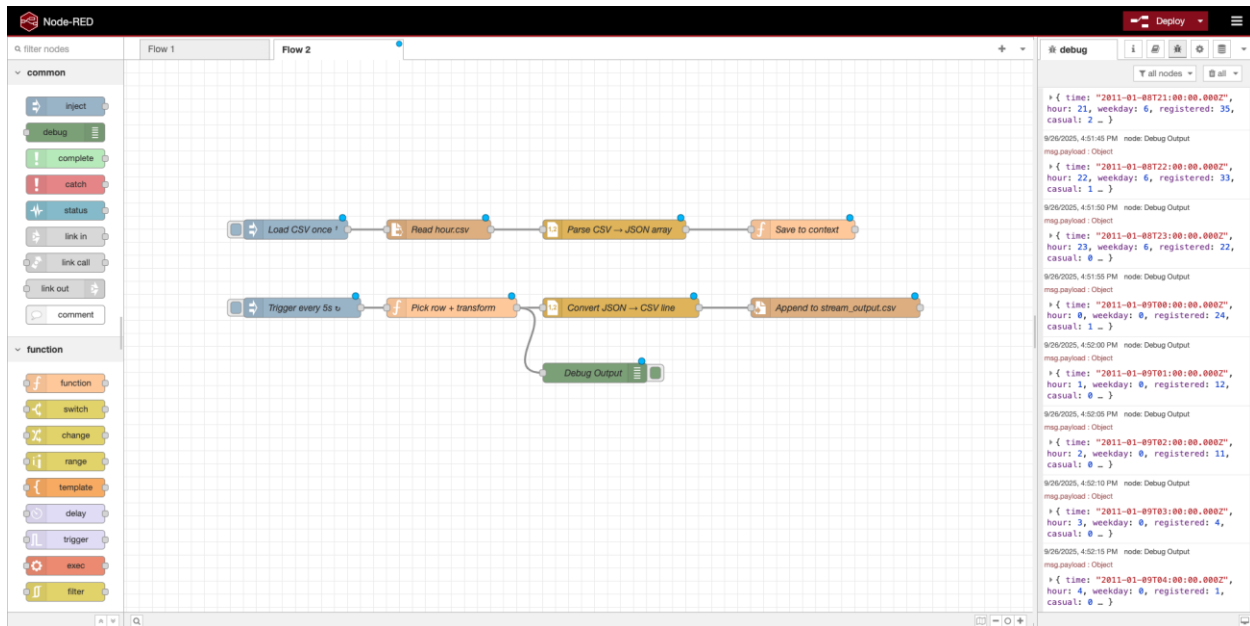
2.1 Batch Pipeline

The batch pipeline reads the entire hour.csv dataset, parses it into JSON, applies feature engineering, and writes the cleaned output into batch_output.csv. An optional debug node was added to preview the first few rows. This pipeline ensures complete coverage of the dataset for historical analysis.



2.2 Streaming Pipeline

The streaming pipeline loads the dataset into memory and sequentially emits one row every 5 seconds. Each row is transformed to include consistency checks and written into `stream_output.csv`, while being visualized in real time via Grafana. The flow context is used to store the current index to guarantee correct sequencing.

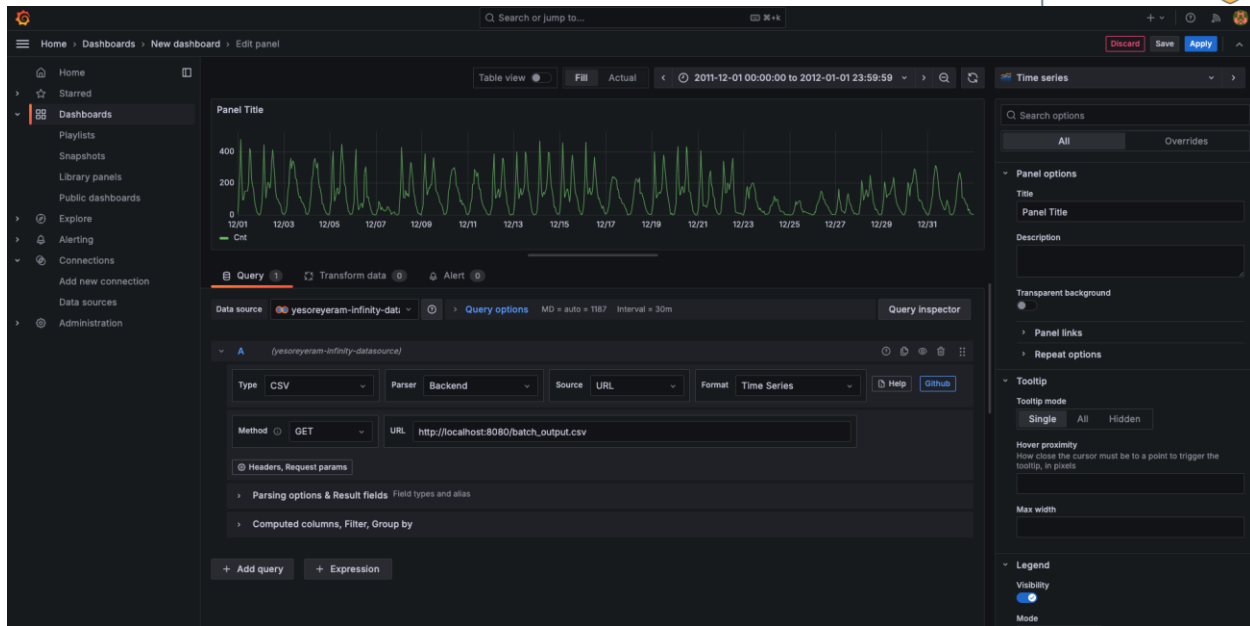


3. Grafana Dashboards

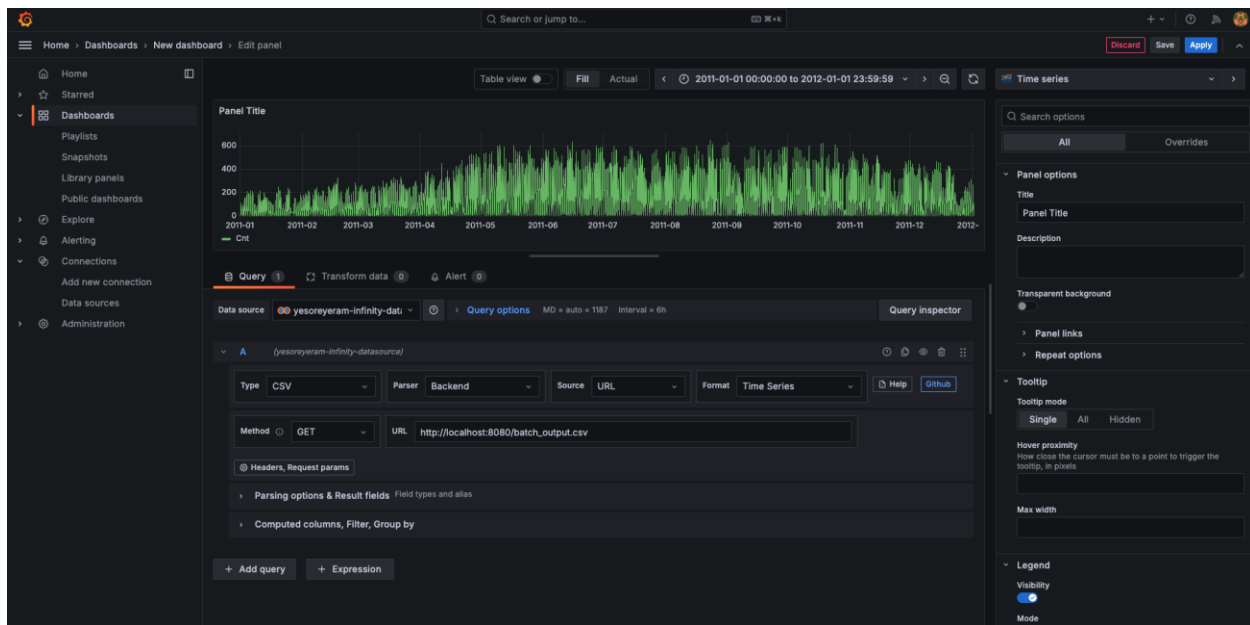
The Infinity plugin accessed the CSV files via **local HTTP path** (served from Node-RED/localhost) rather than directly from the file system, since Grafana by default cannot read local files. This setup ensured the dashboards could refresh continuously as new records were appended.

3.1 Batch Pipeline

This panel plots the rental counts (cnt) across the entire historical dataset (batch_output.csv). It provides a clear view of overall demand trends over time, showing daily and weekly seasonality.

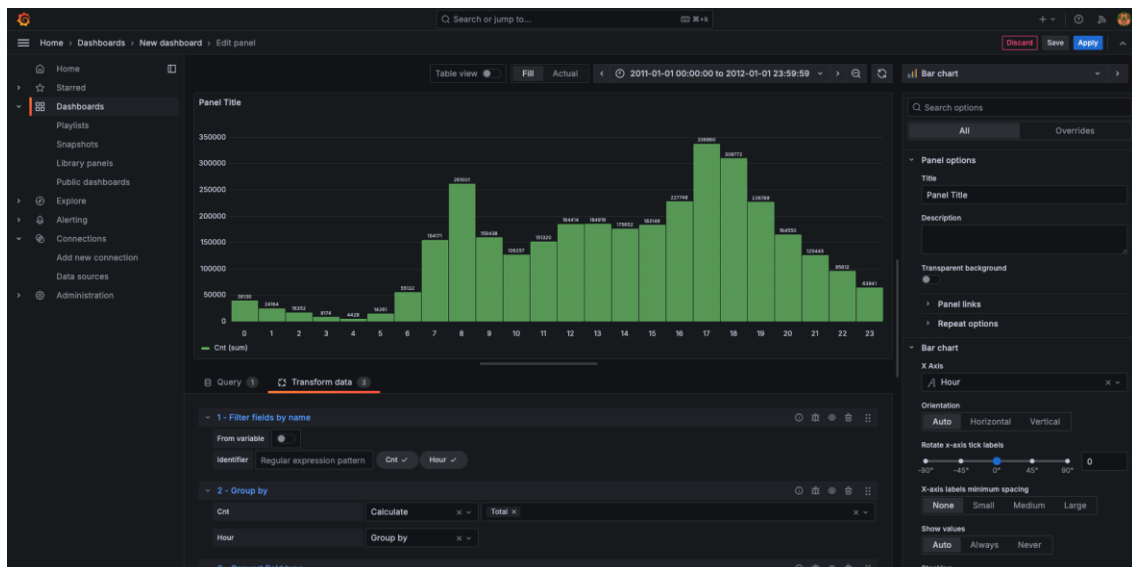
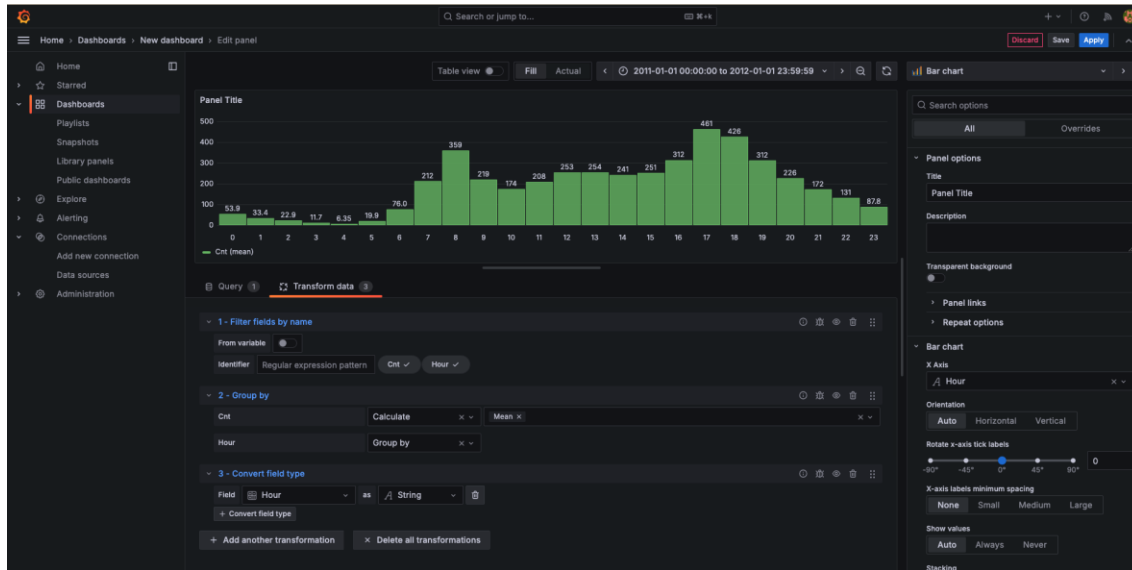


By zooming into the same dataset, we highlight longer-term variations, confirming that the batch pipeline supports stable historical analysis.



Hourly Aggregation:

These panels transform the dataset by grouping counts by hour. The results reveal distinct hourly patterns: peaks during commuting hours (8–9 AM and 5–6 PM) and lower usage during the night.

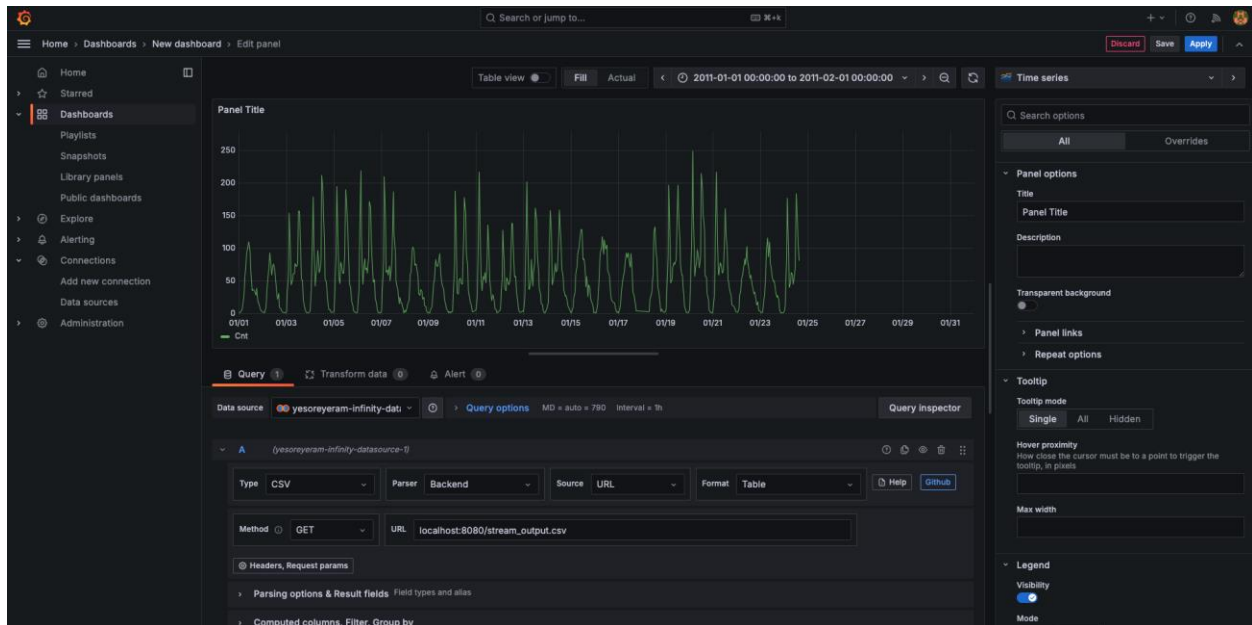


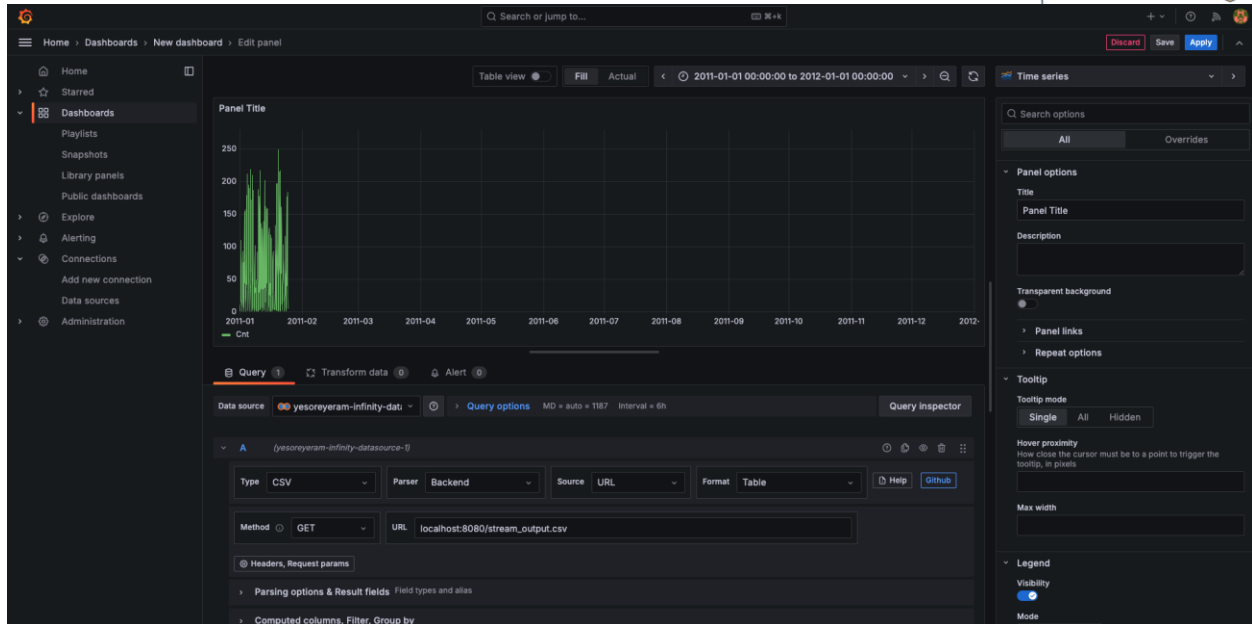
3.2 Streaming Pipeline

This panel displays cnt values streaming every 5 seconds from stream_output.csv. It demonstrates that Grafana can visualize near real-time flows.



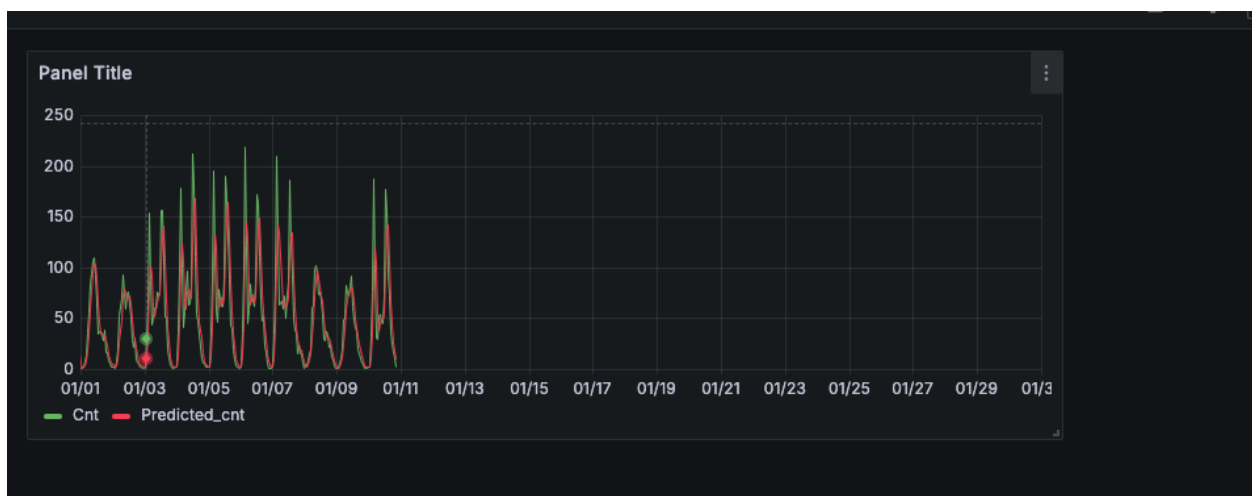
Both actual counts (cnt) and predicted counts (predicted_cnt) are plotted together. This comparison highlights the effect of the moving average predictor implemented in Node-RED, allowing us to observe how predictions align with reality.





To extend the streaming pipeline, we added a **simple prediction stage** in Node-RED. Using a moving average of the past three cnt values, the pipeline generates a new field `predicted_cnt` for each incoming record.

The Grafana panel below overlays both **actual counts (cnt, green)** and **predicted counts (predicted_cnt, red)** on the same time axis. This visualization demonstrates the capability to integrate real-time forecasting directly into the pipeline. While the predictions are simple, they capture short-term trends and provide a baseline for comparing actual vs expected demand.



3.3 Results Overview

The figure below illustrates the integrated Grafana dashboard designed for monitoring both streaming and batch results. The dashboard consists of five panels:

- **Streaming panel (top-left):** compares actual counts with predicted values in near real-time.
- **Batch panel (top-right):** displays long-term historical usage trends across the year.
- **Aggregate by weather (middle-left):** summarizes ridership distribution under different weather situations.
- **Aggregate by season (bottom-left):** shows seasonal variations in demand.
- **Hourly aggregation (bottom-right):** highlights typical daily commuting peaks.

This layout provides an at-a-glance overview, enabling users to simultaneously track real-time predictions, historical patterns, and aggregated statistics.

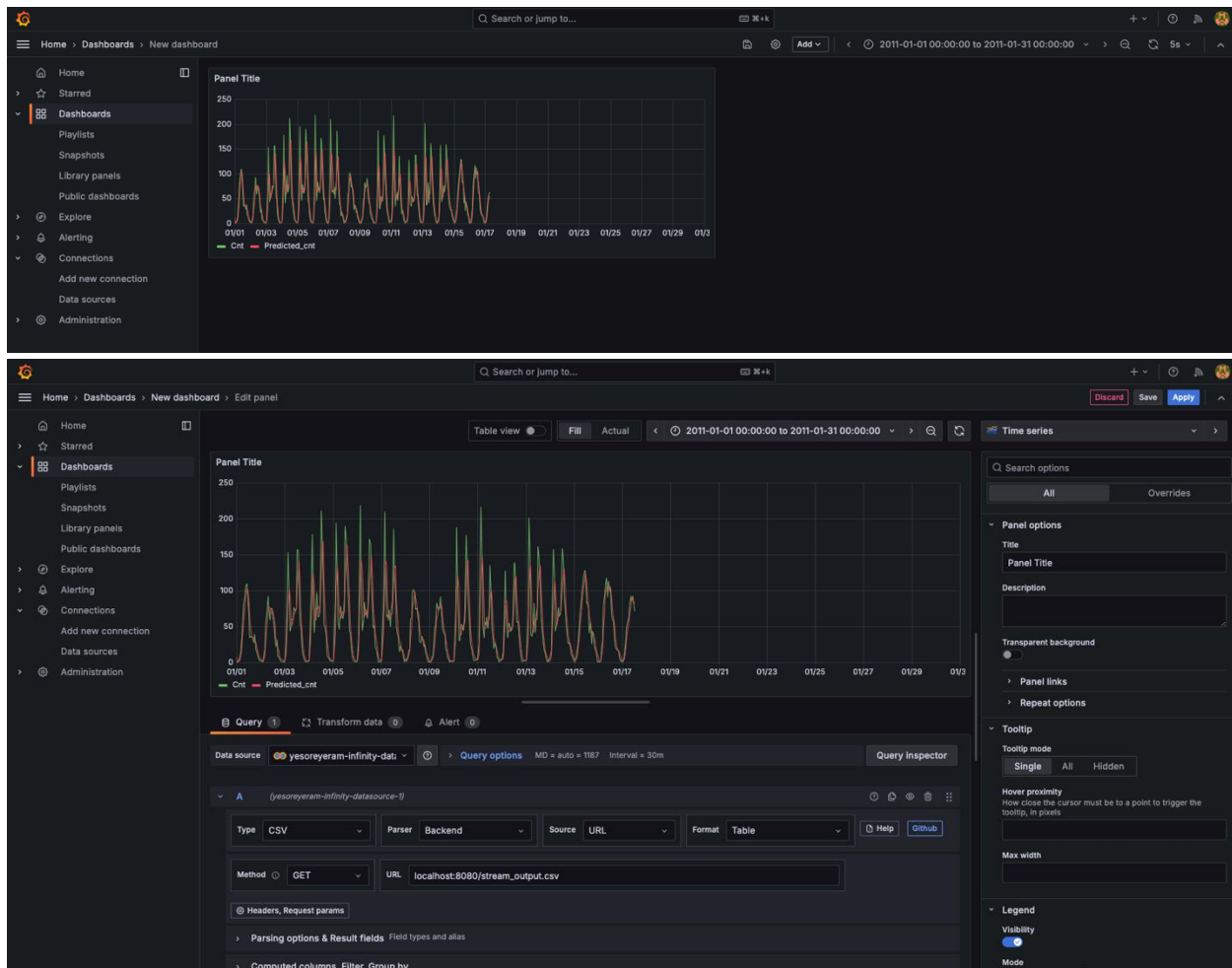


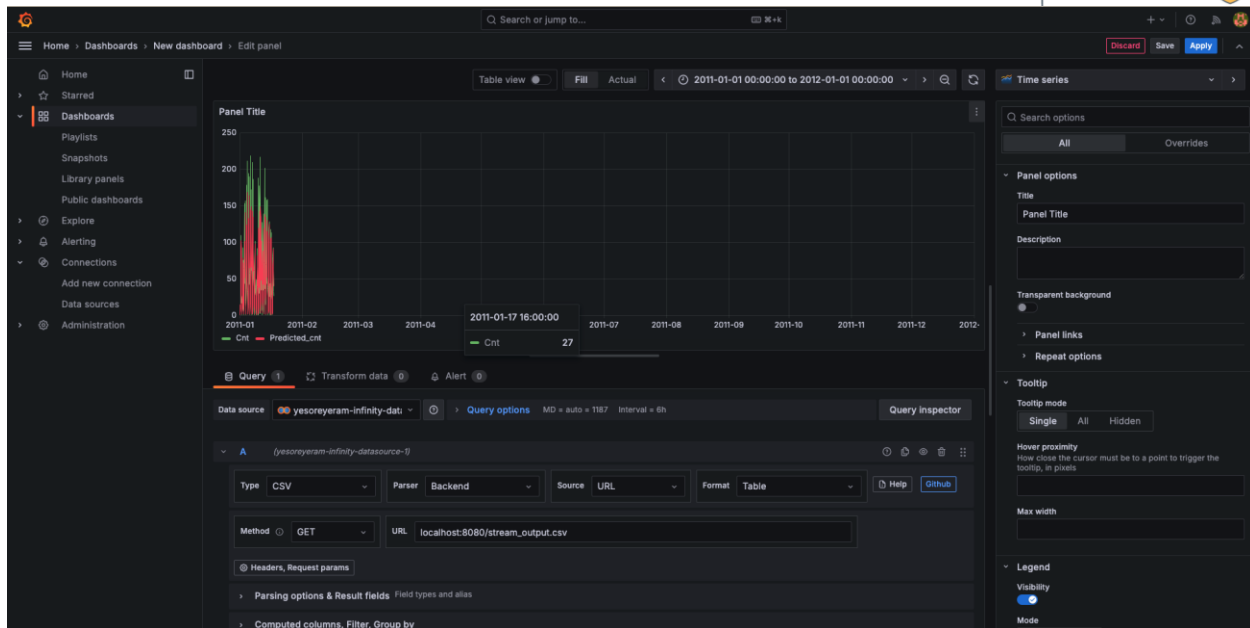
4. Bonus

We extended the streaming pipeline with two enhancements:

4.1 Prediction Stage

A simple moving average over the past three values was implemented in Node-RED to generate the predicted_cnt field. In Grafana, both actual (cnt) and predicted (predicted_cnt) are plotted together for comparison.

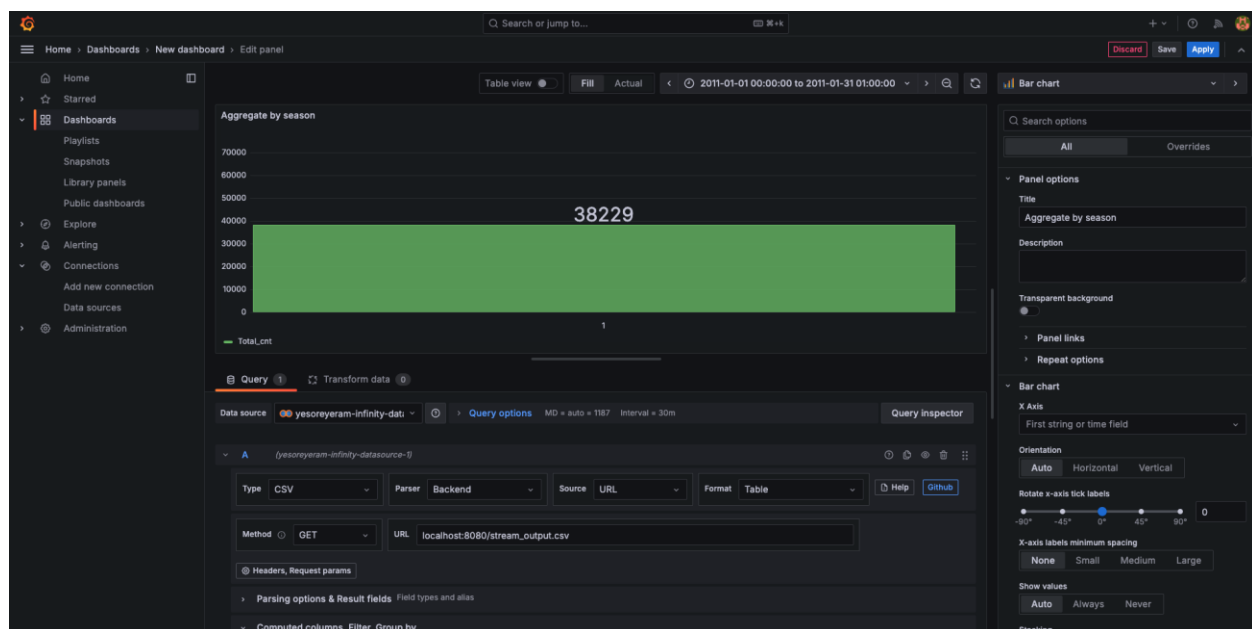
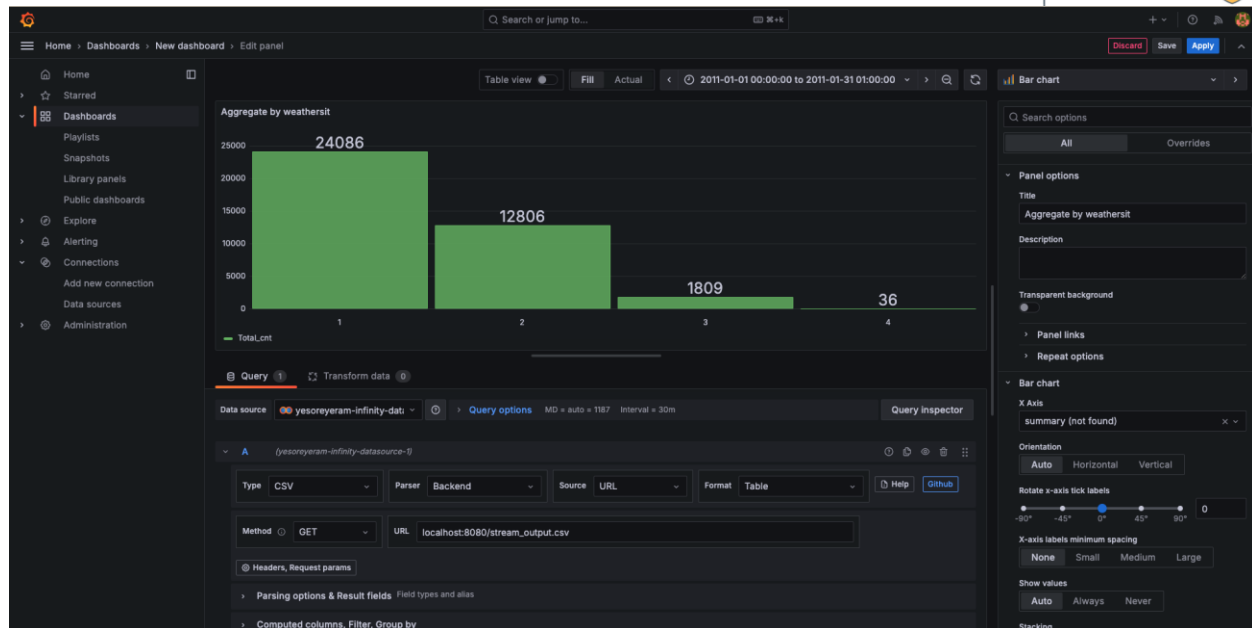




4.2 Aggregation by Season and Weather

In the dataset, both season and weathersit were encoded as numeric values (e.g., 1 = winter, 2 = spring, 3 = summer, 4 = fall). For visualization, these numeric codes were retained in Grafana. Optionally, custom value-to-label mappings can be applied to improve interpretability.





5. Write-up (Main Analysis)

5.1 What are the differences between batch vs streaming pipelines?

The **batch pipeline** processes the entire dataset at once. It is well-suited for historical analysis and guarantees complete coverage, producing consistent outputs such as batch_output.csv. However, batch mode is less responsive because new data cannot be incorporated until the entire batch is re-processed.

The **streaming pipeline**, on the other hand, continuously processes one record at a time (every 5 seconds in our setup). This allows for near real-time visualization in Grafana, making it possible to track trends dynamically and include predictive calculations (e.g., moving average forecasts). The trade-off is that streaming requires careful state management (row index, context storage, etc.) and can encounter issues like lag or missing rows if not synchronized correctly.

5.2 What issues came up?

- **Lag and Synchronization:** The streaming trigger sometimes caused skipped or repeated rows when flow context was not properly managed.
- **Formatting Differences:** Batch output provided a fixed schema, while the streaming output initially missed columns like `predicted_cnt`, `season`, and `weathersit`, requiring additional Function node logic.
- **Visualization Challenges:** Grafana Infinity plugin required explicit column mapping. Initially, only the actual count (`cnt`) was displayed, and the predicted series did not appear until configuration was corrected.
- **Time Offset:** Streaming timestamps were in UTC (`toISOString()`), which created a 5-hour offset relative to local time. This required adjustments to ensure consistent visualization.

5.3 Two suggestions for improvement

- **Introduce Windowed Aggregations in Node-RED:**
Instead of exporting individual rows every 5 seconds, implement a sliding window aggregator (e.g., average demand over the past 1 hour). This would smooth fluctuations, reduce visualization noise, and better support predictive modeling.
- **Automated Data Enrichment for Visualization:**
Extend the pipeline to automatically join contextual attributes (season, weather situation, holidays) during transformation. This avoids manual editing later and makes Grafana dashboards richer, supporting direct comparisons like “Average rentals by weather condition” or “Seasonal demand trends.”

6. Technical Issues & Solutions

During the batch and streaming pipeline implementation with Node-RED and Grafana, several technical challenges were encountered. The following summarizes key issues and their respective solutions:

6.1 Node-RED Installation on macOS

Issue:

Installing Node-RED globally via npm resulted in permission errors, requiring sudo privileges and causing inconsistent environments.

Solution:

Adopted **nvm (Node Version Manager)** to manage Node.js versions. Installed Node-RED in the user environment without sudo, ensuring reproducibility and avoiding system-level conflicts.

6.2 Timestamp Offset in Output

Issue:

Timestamps generated with `toISOString()` appeared 5 hours ahead of expected local time (e.g., 2011-01-01T00:00:00 became 2011-01-01T05:00:00Z). This created misalignment in Grafana visualizations.

Solution:

Constructed timestamps explicitly using `Date.UTC(...)` or exported epoch milliseconds. This ensured consistent time alignment across Node-RED and Grafana.

6.3 CSV Output Format

Issue:

The exported `batch_output.csv` initially contained only one row or had columns in the wrong order, making it incompatible with Grafana.

Solution:

Configured the Node-RED CSV node with an explicit column schema (time, hour, weekday, registered, casual, cnt, cnt_check, mismatch) and enabled header output. This guaranteed a consistent and clean dataset for visualization.

6.4 Grafana Data Source and Visualization

Issue:

Grafana cannot directly load local CSV files by default, and bar chart panels require a valid string or time field.

Solution:

Installed the **Infinity plugin** to load CSV files via HTTP. Applied transformations to cast hour into a string and grouped by hour using SUM(cnt), enabling the required hourly bar charts.

6.5 Streaming Trigger and Context Management

Issue:

The “Trigger every 5s” node initially required manual activation, and the row index did not persist across executions, causing skipped or repeated rows.

Solution:

Utilized Node-RED **flow context (flow.set / flow.get)** to store the current row index. This preserved state across triggers and ensured continuous, sequential streaming without manual intervention.

6.6 Predicted Value Calculation

Issue:

The streaming pipeline initially lacked the predicted_cnt field, preventing visualization of actual vs predicted demand in Grafana.

Solution:

Implemented a **moving average predictor** in a Function node. By storing the last three cnt values in flow context, the node calculated and appended a predicted_cnt field for each record. Grafana could then display dual series (actual vs predicted).

6.7 Adding Season and Weather Attributes

Issue:

The initial streaming output only contained temporal and count values, missing season and weathersit, which were required for aggregation analyses.

Solution:

Modified the transformation Function node to extract season and weathersit from the original dataset and include them in the output schema. This enriched dataset enabled direct aggregation by season and weather within Grafana dashboards.

6.8 Grafana Dual-Series Visualization

Issue:

Grafana initially displayed only the cnt series, making it impossible to compare actual vs predicted counts.

Solution:

Configured the Infinity plugin query editor to map both cnt and predicted_cnt as numeric series with time as the X-axis. Grafana then rendered a dual line chart, clearly showing prediction accuracy over time.

6.9 Aggregation Dashboards by Season and Weather

Issue:

Grafana transformations did not initially support grouping by categorical fields (season, weathersit) when loading from CSV.

Solution:

Applied Grafana's **Transform → Group by** feature, casting season/weather fields to strings and applying SUM(cnt) or AVG(cnt). This enabled bar and stacked bar charts that highlighted seasonal demand and weather-related rental patterns.

6.10 CSV Header Recognition in Grafana Infinity

Issue:

When loading the CSV file in Grafana using the Infinity plugin, the field names were not recognized correctly. Without proper headers, Grafana displayed generic column names (e.g., Column1, Column2), making transformations and queries difficult.

Solution:

Enabled the **"First row as header"** option in the Infinity query editor. This ensured that fields such as time, cnt, and predicted_cnt were correctly identified, allowing proper mapping in Grafana panels.

7. Reflection

Implementing both batch and streaming pipelines with Node-RED and Grafana provided valuable hands-on experience with real-time data processing. The project highlighted that while batch pipelines are straightforward and stable for historical analysis, streaming pipelines require careful design to handle context, state, and continuous updates.

A major takeaway was the importance of data schema consistency: even small formatting mismatches between batch and stream outputs can cause difficulties in downstream visualization. This reinforced the need for clear data contracts between pipeline stages.

Another learning point was how visualization tools (Grafana) interact with data. Although powerful, Grafana requires precise configuration (column mapping, transformations, and refresh intervals) to display the intended insights. The process of enabling predicted values and grouping by season/weather improved my understanding of how dashboards can evolve from simple line charts into decision-support tools.

Overall, this exercise strengthened our skills in integrating low-code tools like Node-RED with visualization platforms and gave us confidence in troubleshooting common issues such as timestamp offsets, CSV formatting, and plugin limitations. If given more time, we would explore integrating external APIs (e.g., weather forecasts) to enhance predictive modeling and provide more actionable insights.